CS401- Computer Architecture and Assembly Language Programming

Solution to Assignment # 1

Spring 2009

Total Marks: 20

Solution

Question_1:

What is the effective address generated by each of the following instruction?
Initially BX=0x0010, label=0x0124, [label]=0x0001, and SI=0x00F1
(Offsets in part a, b and f are in decimal)

a. mov ax, [bx+30]
   Effective Address = bx + 30
   = 0010 + 001E
   = 002E

b. mov ax, [bx+20]
   Effective Address = bx + 20
   = 0010 + 0014
   = 0024

c. mov ax, [bx+label]
   Effective Address = bx + label
   = 0010 + 0124
   = 0134

d. mov ax, [label+bx]
   Effective Address = label + bx
   = 0124 + 0010
   = 0134

e. mov ax, [bx+si]
   Effective Address = bx + si
   = 0010 + 00F1
   = 0101

f. mov ax, [si+15]
   Effective Address = si + 15
   = 00F1 + 000F
   = 0100

Question_2:

Calculate the physical memory address generated by the following segment offset pairs (both are hexadecimal values).

a. 0000:0100
   16-bit Segment Address: 0000
   16-bit Offset Address: 0100

Step1: Converting these to 20-bit addresses
20-bit Segment Address: 00000
20-bit Offset Address: 00100
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): 00000 + 00100 = 00100
20-bit Physical address (in binary): 0000 0000 0001 0000 0000
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

b. 0010:0000
16-bit Segment Address: 0010
16-bit Offset Address: 0000
Step1: Converting these to 20-bit addresses
20-bit Segment Address: 00100
20-bit Offset Address: 00000
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): 00100 + 00000 = 00100
20-bit Physical address (in binary): 0000 0000 0001 0000 0000
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

c. 1DAD:1234
16-bit Segment Address: 1DAD
16-bit Offset Address: 1234
Step1: Converting these to 20-bit addresses
20-bit Segment Address: 1DAD0
20-bit Offset Address: 01234
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): 1DAD0 + 01234 = 1ED04
20-bit Physical address (in binary): 0001 1110 1101 0000 0100
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

d. 4321:FFFF
16-bit Segment Address: 4321
16-bit Offset Address: FFFF
Step1: Converting these to 20-bit addresses
20-bit Segment Address: 43210
20-bit Offset Address: 0FFFF
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): 43210 + 0FFFF = 5320F
20-bit Physical address (in binary): 0101 0011 0010 0000 1111
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

e. FFFF:4321
16-bit Segment Address: FFFF

16-bit Offset Address: 4321
Step1: Converting these to 20-bit addresses
20-bit Segment Address: FFFF0
20-bit Offset Address: 04321
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): FFFF0 + 04321 = 104311 [As it is more than 20-bits, so wrap it around, that is, discard the most significant hexadecimal digit]
20-bit Physical address (in binary): 0000 0100 0011 0001 0001
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

f. FFEF:4421
16-bit Segment Address: FFEF
16-bit Offset Address: 4421
Step1: Converting these to 20-bit addresses
20-bit Segment Address: FFEF0
20-bit Offset Address: 04421
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): FFEF0 + 04421 = 104311 [As it is more than 20-bits, so wrap it around, that is, discard the most significant hexadecimal digit]
20-bit Physical address (in binary): 0000 0100 0011 0001 0001
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

g. 1080:0200
16-bit Segment Address: 1080
16-bit Offset Address: 0200
Step1: Converting these to 20-bit addresses
20-bit Segment Address: 10800
20-bit Offset Address: 00200
Step2: Performing hexadecimal addition:
20-bit Physical address (in hexadecimal): 10800 + 00200 = 10A00
20-bit Physical address (in binary): 0001 0000 1010 0000 0000
[Each hexadecimal digit can be represented in its 4-bit equivalent binary (as shown above)]

Question_3:

- Assembly language code:

```
;Program to add first five odd numbers without defining these numbers.
        [ORG 0x100]
            mov ax,0      ; initialize AX register with value 0
            mov bx,1      ; initialize BX register with value 1
            mov cx,5      ; We have to sum 5 numbers so initialize CX with
        value 5
```

```
Label:  add ax,bx     ; add BX contents to AX
        add bx,2      ; add 2 to BX to make it next odd number
        sub cx,1      ; subtract 1 from CX
        jnz Label     ; Jump if not zero, that is, we have to add more
numbers

        mov ax, 0x4C00 ; terminate program
        int 0x21
```

Ø Screen-shot displaying result in AX register (On AFD window, press the key combination Alt + PrintScreen to take screen-shot and paste it here)

1+3+5+7+9 = 25 (decimal) = 19 (hexadecimal)